Apuntes examen: Git y GitHub Pages

Chuleta con ejemplos de todo lo que entra: `config`, `init`, `clone`, `add`, `commit`, `push`, `branch`, `merge`, `reset`, `revert`, `tag`, `.gitignore` y GitHub Pages. (Actions va aparte.)

1. Configuración inicial (una sola vez)

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu@email.com"
git config --list          # comprobar
```

Sin esto, `git commit` te pedirá quién eres.

2. Las tres zonas

```
Directorio de trabajo  --git add-->  Staging  --git commit-->  Repositorio
local  --git push-->  GitHub (origin)
```

3. Crear un repositorio

Opción A: empiezo en local y luego lo subo a GitHub

```
mkdir actions
cd actions
git init
echo "# Mi proyecto" > README.md
git add .
git commit -m "primer commit"
```

Creo en GitHub un repo **vacío** llamado `actions` (sin README) y lo enlazo:

```
git remote add origin https://github.com/MI_USUARIO/actions.git
git branch -M main
git push -u origin main
```

- `remote add origin URL`: da el nombre `origin` a la URL remota.
- `branch -M main`: renombra la rama actual a `main`.
- `push -u origin main`: sube y deja enlazada la rama (después basta con `git push`).

Opción B: empiezo en GitHub y lo traigo (clone)

```
git clone https://github.com/titansmc/actions.git
cd actions
```

`clone` ya configura `origin` automáticamente.

Caso típico del profesor: clonar un repo ajeno y subirlo a mi cuenta

```
git clone https://github.com/titansmc/actions.git
cd actions
git remote set-url origin https://github.com/MI_USUARIO/actions.git
git push -u origin main
```

(Antes hay que crear el repo `actions` vacío en mi GitHub.) Ver a dónde apunta: `git remote -v`.

4. Ciclo diario: status, add, commit, push, pull

```
git status                      # qué ha cambiado
git add requirements.txt        # añade un archivo al staging
git add .                       # añade todo
git commit -m "añado requirements.txt"
git push                        # sube a GitHub
git pull                        # baja cambios de GitHub
```

Ver historial y diferencias:

```
git log                         # historial completo
git log --oneline               # una línea por commit (muestra los ids
cortos)
git diff                        # cambios aún sin añadir
git diff --staged               # cambios ya añadidos, sin commitear
```

Atajo: `git commit -am "mensaje"` añade y commitea los archivos **ya rastreados** (no sirve para archivos nuevos).

5. Ramas (branch)

```
git branch                        # lista ramas (* = la actual)
git branch test                   # crea la rama test
git checkout test                 # cambia a test   (o: git switch test)
git checkout -b development       # crea y cambia a la vez (o: git switch -c
development)
```

Flujo completo de ejemplo (rama `test` con un cambio):

```
git checkout -b test
# edito el archivo...
git add .
git commit -m "python 3.12 en la rama test"
git push -u origin test           # primera vez que subo la rama
```

Fusionar (merge): me pongo en la rama que **recibe** y fusiono la otra.

```
git checkout main
git merge test                    # trae los cambios de test a main
git push
```

Borrar:

```
git branch -d test                # borra la rama local (ya fusionada)
git branch -D test                # fuerza el borrado
git push origin --delete test     # borra la rama en GitHub
```

Nota: cada rama tiene su propia versión de los archivos. Al cambiar de rama, los archivos de tu carpeta cambian.

6. Deshacer cambios

Antes de hacer commit

```
git restore archivo.txt           # descarta cambios no añadidos (¡se
pierden!)
git restore --staged archivo.txt # saca el archivo del staging, conserva los
cambios
```

reset: mueve la rama hacia atrás (reescribe historial)

```
git reset --soft HEAD~1          # quita el último commit, cambios quedan en staging
git reset --mixed HEAD~1         # quita el commit, cambios quedan sin añadir (por defecto)
git reset --hard HEAD~1          # quita el commit y BORRA los cambios
git reset --hard a1b2c3d         # vuelve a un commit concreto (id de git log --oneline)
```

`HEAD~1` = un commit antes del actual; `HEAD~3` = tres antes.

Si ya hiciste `push` y quieres que GitHub lo refleje, necesitas `git push --force` (peligroso). Por eso, tras un push, mejor `revert`.

revert: crea un commit nuevo que deshace otro (conserva historial)

```
git log --oneline                # busco el id del commit a deshacer
git revert a1b2c3d               # crea un commit inverso
git push
```

Comparativa

	`reset`	`revert`
Qué hace	Borra commits del historial	Añade un commit que deshace otro
Historial	Se reescribe	Se conserva
Tras `push`	Peligroso (`--force`)	Seguro
Cuándo usarlo	Commits locales, aún sin subir	Commits ya subidos

7. Tags (versiones)

```
git tag v1.0                          # tag ligero sobre el commit actual
git tag -a v1.0 -m "Versión 1.0"      # tag anotado (con mensaje)
git tag v0.9 a1b2c3d                  # tag sobre un commit antiguo
git tag                               # listar tags
git show v1.0                         # ver info del tag
git push origin v1.0                  # subir UN tag
git push origin --tags                # subir todos
git tag -d v1.0                       # borrar tag local
git push origin --delete v1.0         # borrar tag en GitHub
```

Ojo: `git push` normal NO sube los tags. Hay que subirlos a mano.

8. .gitignore

Archivo `.gitignore` en la raíz del repo:

```
# Python
__pycache__/
*.pyc
.venv/

# PyInstaller
dist/
build/
*.spec

# Secretos
.env

# Un archivo concreto
notas.txt
```

```
git add .gitignore
git commit -m "añado .gitignore"
```

Si el archivo **ya estaba subido**, ignorarlo no basta; hay que dejar de rastrearlo:

```
git rm --cached archivo.txt        # un archivo
git rm -r --cached carpeta/        # una carpeta
git commit -m "dejo de rastrear archivo.txt"
```

9. GitHub Pages

Publica una web estática (HTML/CSS/JS) desde un repo.

Paso a paso

```
git clone https://github.com/MI_USUARIO/mi-web.git
cd mi-web
```

Creo un `index.html`:

```
<!DOCTYPE html>
<html>
  <head><title>Mi web</title></head>
  <body><h1>Hola desde GitHub Pages</h1></body>
</html>
```

```
git add index.html
git commit -m "añado index.html"
git push
```

En GitHub:

1. Repo → **Settings** → **Pages** (menú izquierdo).
2. **Source**: *Deploy from a branch*.
3. **Branch**: `main` y carpeta `/ (root)` (o `/docs`) → **Save**.
4. Esperar 1-2 minutos. URL: `https://MI_USUARIO.github.io/mi-web/`

Detalles importantes

- Cada `push` a la rama publicada actualiza la web.
- El archivo de entrada tiene que llamarse `index.html`.
- Si publicas desde `/docs`, el `index.html` debe estar dentro de `docs/`.
- Puedes publicar desde otra rama (por ejemplo `gh-pages`) eligiéndola en Settings → Pages.
- Si sale 404, revisa: nombre `index.html`, rama/carpeta elegidas y espera un poco.

10. Simulacro de ejercicio completo

Enunciado: crear un repo, subir una web, crear una rama con un cambio, etiquetar versión y publicar.

```
# 1. Repo local
mkdir web && cd web
git init
echo "<h1>Hola</h1>" > index.html
git add .
git commit -m "primera versión"

# 2. Enlazar con GitHub (repo vacío ya creado) y subir
git remote add origin https://github.com/MI_USUARIO/web.git
git branch -M main
git push -u origin main
```

```
# 3. Rama nueva con un cambio
git checkout -b development
echo "<p>Más contenido</p>" >> index.html
git add .
git commit -m "añado párrafo"
git push -u origin development

# 4. Fusionar en main
git checkout main
git merge development
git push

# 5. Tag de versión
git tag -a v1.0 -m "Versión 1.0"
git push origin v1.0

# 6. GitHub Pages: Settings -> Pages -> main / (root)
```

11. Errores típicos y solución

Problema	Causa / solución
`fatal: not a git repository`	No estás dentro de la carpeta del repo, o falta `git init`
`Author identity unknown`	Falta `git config user.name` / `user.email`
`remote origin already exists`	Usa `git remote set-url origin URL`
`push` rechazado (rejected)	El remoto tiene commits que no tienes: `git pull` y luego `git push`
Autenticación fallida al hacer push	GitHub no acepta contraseña: usa un **Personal Access Token** como contraseña o SSH
El tag no aparece en GitHub	Falta `git push origin v1.0`
`.gitignore` no ignora un archivo	Ya estaba rastreado: `git rm --cached archivo`
Hice `reset --hard` y perdí cambios	Mira `git reflog` para encontrar el commit anterior y vuelve con `git reset --hard ID`
Estoy en una rama equivocada	`git branch` para verlo, `git checkout nombre` para cambiar

12. Resumen de una página

```
git init | git clone URL
git status | git log --oneline | git diff
git add . | git commit -m "msg"
git remote add origin URL | git remote -v
git push -u origin RAMA | git push | git pull
git branch | git checkout -b RAMA | git merge RAMA
git reset --soft/--mixed/--hard HEAD~1 | git revert ID
git tag -a vX -m "msg" | git push origin vX
.gitignore | git rm --cached archivo
Settings -> Pages -> Deploy from a branch
```